Binary Search — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

48. Binary Search to Find X in Sorted Array

Understanding the Problem NEW

You're given a sorted array and a value X, and you need to check whether X exists in it. Because the array is sorted, you don't have to check every element one by one — think of how you'd look up a word in a paper dictionary: you don't start at page 1, you flip to roughly the middle and decide whether to go left or right from there.

Brute Force Approach NEW

The naive way to search any array — sorted or not — is **Linear Search**: check every element from left to right until you find X or reach the end. That's O(n) time, and critically, it never makes use of the fact the array is sorted.

Why We Optimize NEW

Because the array is sorted, comparing X to the **middle** element instantly tells you which half X could possibly be in — if X is bigger than the middle, it can only be in the right half; if smaller, only the left half. You can then throw away the other half entirely and repeat on the remaining half. Each comparison eliminates half of the remaining search space, so instead of checking n elements one at a time, you check only about $\log_2 n$ of them — turning O(n) into O(log n). This is the single most important idea in this whole chapter, and every later problem is a variation of it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find if element X exists in a sorted array using binary search.

Visual Diagram

```
Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91], Target: 23

        Step 1: low=0, high=9, mid=4
        [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
                          ↑
                        mid=16
        16 < 23 → Search right half
```

```
        Step 2: low=5, high=9, mid=7
        [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
                              ↑
                            mid=56
        56 > 23 → Search left half

        Step 3: low=5, high=6, mid=5
        [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
                              ↑
                            mid=23
        23 == 23 → Found at index 5

        Result: Index 5
```

Solution Approach

Concept: Divide and conquer - compare with middle element.

Algorithm

```
Algorithm BINARY_SEARCH(arr, target):
           low = 0, high = n-1
           While low <= high:
               mid = (low + high) / 2
               If arr[mid] == target:
                   Return mid
               Else If arr[mid] < target:
                   low = mid + 1
               Else:
                   high = mid - 1
           Return -1
```

Java Implementation

```
class Solution {
           public int binarySearch(int[] arr, int target) {
               int low = 0, high = arr.length - 1;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (arr[mid] == target) {
                         return mid;
                     } else if (arr[mid] < target) {
                         low = mid + 1;
                     } else {
                         high = mid - 1;
                     }
                 }
```

```
                return -1;
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

49. Implement Lower Bound

Understanding the Problem NEW

Find the **first index** where you could insert the target value while keeping the array sorted — equivalently, the first position holding a value that is greater than or equal to the target. E.g. in `[1,2,2,3]` the lower bound of 2 is index 1.

Brute Force Approach NEW

Scan the array left to right and stop at the first element ≥ target — O(n) time, ignoring the sorted structure that's available to exploit.

Why We Optimize NEW

This is standard Binary Search with a small twist: instead of stopping the moment you find a match, you keep searching the **left half** even after finding an element equal to the target, because there might be an earlier occurrence. You track the best (leftmost) qualifying index found so far and keep narrowing. Same O(log n) benefit as basic binary search, just with a modified 'keep looking left' rule.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the first position where target can be inserted to maintain sorted order (first >= target).

Visual Diagram

```
Array: [1, 2, 3, 3, 5, 7, 8], Target: 3

       Step 1: low=0, high=6, mid=3
       [1, 2, 3, 3, 5, 7, 8]
                ↑
              arr[3]=3
       3 >= 3 → ans=3, high=2

       Step 2: low=0, high=2, mid=1
       [1, 2, 3, 3, 5, 7, 8]
             ↑
           arr[1]=2
       2 < 3 → low=2
```

```
Step 3: low=2, high=2, mid=2
[1, 2, 3, 3, 5, 7, 8]
          ↑
        arr[2]=3
3 >= 3 → ans=2, high=1

Result: 2 (first position where 3 can be inserted)

Target: 4 → Lower Bound = 4 (position after 3s)
Target: 0 → Lower Bound = 0
Target: 9 → Lower Bound = 7 (length)
```

Solution Approach

Concept: Find first element >= target.

Algorithm

```
Algorithm LOWER_BOUND(arr, target):
           low = 0, high = n
           ans = n
           While low < high:
               mid = (low + high) / 2
               If arr[mid] >= target:
                   ans = mid
                   high = mid
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int lowerBound(int[] arr, int target) {
               int low = 0, high = arr.length;
               int ans = arr.length;

                 while (low < high) {
                     int mid = low + (high - low) / 2;

                     if (arr[mid] >= target) {

                       ans = mid;
                       high = mid;
                   } else {
                       low = mid + 1;
                   }
               }
               return ans;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

50. Implement Upper Bound

Understanding the Problem NEW

Very similar to lower bound, but now find the first index whose value is **strictly greater than** the target (not equal to it). In `[1,2,2,3]`, the upper bound of 2 is index 3 (where the 3 sits).

Brute Force Approach NEW

Same naive idea as before — scan until you find the first element strictly greater than target, O(n).

Why We Optimize NEW

Again a binary search variant: when the middle element is less than *or equal to* the target, you now search right (since equal values don't qualify for 'strictly greater'); when it's greater, you search left but remember this position as a candidate. The only change from Lower Bound is the comparison operator, but that single tweak changes what boundary you land on — it's worth noticing how small a change in the binary search condition can redirect it to solve a related but distinct problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the first position where target can be inserted (first > target).

Visual Diagram

```
Array: [1, 2, 3, 3, 5, 7, 8], Target: 3

       Step 1: low=0, high=6, mid=3
       [1, 2, 3, 3, 5, 7, 8]
                ↑
              arr[3]=3
       3 > 3? No → low=4

       Step 2: low=4, high=6, mid=5
       [1, 2, 3, 3, 5, 7, 8]
                     ↑
                   arr[5]=7
```

```
        7 > 3 → ans=5, high=4

        Step 3: low=4, high=4, mid=4
        [1, 2, 3, 3, 5, 7, 8]
                    ↑
                  arr[4]=5
        5 > 3 → ans=4, high=3

        Result: 4 (first position > 3)

        Target: 6 → Upper Bound = 5 (position of 7)
        Target: 0 → Upper Bound = 0
        Target: 9 → Upper Bound = 7 (length)
```

Solution Approach

Concept: Find first element > target.

Algorithm

```
Algorithm UPPER_BOUND(arr, target):
            low = 0, high = n
            ans = n
            While low < high:
                mid = (low + high) / 2
                If arr[mid] > target:
                    ans = mid
                    high = mid
                Else:
                    low = mid + 1
            Return ans
```

Java Implementation

```
class Solution {
            public int upperBound(int[] arr, int target) {
                int low = 0, high = arr.length;
                int ans = arr.length;

                  while (low < high) {
                      int mid = low + (high - low) / 2;

                      if (arr[mid] > target) {

                        ans = mid;
                        high = mid;
                    } else {
                        low = mid + 1;
                    }
                }
```

```
                return ans;
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

51. Search Insert Position

Understanding the Problem NEW

Given a sorted array and a target, find the index where the target is (if present) or the index where it would be inserted to keep the array sorted (if absent). This is essentially the Lower Bound problem in disguise.

Brute Force Approach NEW

Linear scan comparing each element until you pass the target's rightful position — O(n).

Why We Optimize NEW

This problem *is* Lower Bound: the first index where `arr[i] >= target` is exactly the answer, whether target is present or not. Recognizing that two differently-worded problems are secretly the same problem is a valuable skill — it means you already have an O(log n) solution ready to reuse instead of designing something new.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find position where target should be inserted in sorted array.

Visual Diagram

```
Array: [1, 3, 5, 6], Target: 5

       Search:
       Step 1: low=0, high=3, mid=1
       [1, 3, 5, 6]
           ↑
         arr[1]=3
       3 < 5 → low=2

       Step 2: low=2, high=3, mid=2
       [1, 3, 5, 6]
                ↑
              arr[2]=5
       5 == 5 → Return 2
```

```
Result: 2

Array: [1, 3, 5, 6], Target: 2
Step 1: mid=1, arr[1]=3 > 2 → high=1
Step 2: mid=0, arr[0]=1 < 2 → low=1
Result: 1 (insert between 1 and 3)

Array: [1, 3, 5, 6], Target: 7
Search through all → low=4
Result: 4 (insert at end)

Array: [1, 3, 5, 6], Target: 0
Search → high becomes -1
Result: 0 (insert at beginning)
```

Solution Approach

Concept: Binary search for insertion point.

Algorithm

```
Algorithm SEARCH_INSERT(arr, target):
           low = 0, high = n-1
           While low <= high:
               mid = (low + high) / 2
               If arr[mid] == target:
                   Return mid
               Else If arr[mid] < target:
                   low = mid + 1
               Else:
                   high = mid - 1
           Return low
```

Java Implementation

```
class Solution {
           public int searchInsert(int[] nums, int target) {
               int low = 0, high = nums.length - 1;

                 while (low <= high) {

                   int mid = low + (high - low) / 2;

                   if (nums[mid] == target) {
                       return mid;
                   } else if (nums[mid] < target) {
                       low = mid + 1;
                   } else {
                       high = mid - 1;
```

```
                }
            }
            return low;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

52. Floor/Ceil in Sorted Array

Understanding the Problem NEW

Floor is the largest value in the array that is ≤ target; Ceil is the smallest value that is ≥ target. E.g. in `[1,2,8,10,11]` with target 5: floor is 2, ceil is 8.

Brute Force Approach NEW

Scan the whole array once tracking the closest value below the target and the closest value above it — O(n), correct but ignores sortedness.

Why We Optimize NEW

Floor and Ceil are just **two separate binary searches** with a slightly adjusted condition each time — Ceil is close to the Lower Bound idea (first element ≥ target), and Floor is its mirror image searching from the other direction (last element ≤ target). Once you've internalized the Lower/Upper Bound pattern, Floor/Ceil become almost free — same log n technique, just pointed at slightly different boundaries.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find floor (largest <= target) and ceil (smallest >= target).

Visual Diagram

```
Array: [2, 3, 5, 9, 14, 16, 18], Target: 15

       Floor (largest ≤ 15):
       [2, 3, 5, 9, 14, 16, 18]
                   ↑
              floor = 14

       Ceil (smallest ≥ 15):
       [2, 3, 5, 9, 14, 16, 18]
                      ↑
                 ceil = 16

       Target: 10
       Floor: 9, Ceil: 14
```

```
Target: 1
Floor: -∞, Ceil: 2

Target: 20
Floor: 18, Ceil: ∞
```

Solution Approach

Concept: Binary search for both floor and ceil.

Algorithm

```
Algorithm FLOOR_CEIL(arr, target):
           floor = -∞, ceil = ∞
           low = 0, high = n-1

            While low <= high:
                mid = (low + high) / 2
                If arr[mid] == target:
                    Return {arr[mid], arr[mid]}
                Else If arr[mid] < target:
                    floor = arr[mid]
                    low = mid + 1
                Else:
                    ceil = arr[mid]
                    high = mid - 1

            Return {floor, ceil}
```

Java Implementation

```
class Solution {
           public int[] floorAndCeil(int[] arr, int target) {
               int floor = -1, ceil = -1;
               int low = 0, high = arr.length - 1;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (arr[mid] == target) {

                       return new int[]{arr[mid], arr[mid]};
                   } else if (arr[mid] < target) {
                       floor = arr[mid];
                       low = mid + 1;
                   } else {
                       ceil = arr[mid];
                       high = mid - 1;
                   }
```

```
                }
                return new int[]{floor, ceil};
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

53. Find First or Last Occurrence of a Number

Understanding the Problem NEW

Find both the first and the last position at which a target value occurs in a sorted array that may contain duplicates. E.g. in `[5,7,7,8,8,10]`, target 8 occurs at first index 3 and last index 4.

Brute Force Approach NEW

Scan the array once left to right to find the first occurrence, and once right to left (or continue the same scan) to find the last — O(n), fine, but the sorted order makes something faster possible.

Why We Optimize NEW

First occurrence is exactly Lower Bound of the target (the first index ≥ target, verified to actually equal target); last occurrence is Upper Bound of the target, minus one. Reusing the Lower/Upper Bound binary searches gets both answers in O(log n) total, instead of a full O(n) scan.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find first and last occurrence of target in sorted array.

Visual Diagram

```
Array: [2, 4, 4, 4, 6, 8, 10], Target: 4

       First Occurrence:
       Step 1: mid=3, arr[3]=4 → first=3, high=2
       Step 2: mid=1, arr[1]=4 → first=1, high=0
       Step 3: mid=0, arr[0]=2 < 4 → low=1
       Result: first=1

       Last Occurrence:
       Step 1: mid=3, arr[3]=4 → last=3, low=4
       Step 2: mid=5, arr[5]=8 > 4 → high=4
       Step 3: mid=4, arr[4]=6 > 4 → high=3
       Result: last=3
```

```
Range: [1, 3]
```

Solution Approach

Concept: Binary search with modified condition.

Algorithm

```
Algorithm FIRST_OCCURRENCE(arr, target):
          low = 0, high = n-1
          ans = -1
          While low <= high:
              mid = (low + high) / 2
              If arr[mid] == target:
                   ans = mid
                   high = mid - 1
              Else If arr[mid] < target:
                   low = mid + 1
              Else:
                   high = mid - 1
          Return ans

      Algorithm LAST_OCCURRENCE(arr, target):
          low = 0, high = n-1
          ans = -1
          While low <= high:
              mid = (low + high) / 2
              If arr[mid] == target:
                   ans = mid
                   low = mid + 1
              Else If arr[mid] < target:
                   low = mid + 1
              Else:
                   high = mid - 1
          Return ans
```

Java Implementation

```
class Solution {
          public int[] searchRange(int[] nums, int target) {
              int first = findFirst(nums, target);

              int last = findLast(nums, target);
              return new int[]{first, last};
          }

          private int findFirst(int[] nums, int target) {
              int low = 0, high = nums.length - 1;
              int ans = -1;
```

```
            while (low <= high) {
                int mid = low + (high - low) / 2;

                if (nums[mid] == target) {
                    ans = mid;
                    high = mid - 1;
                } else if (nums[mid] < target) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return ans;
        }

        private int findLast(int[] nums, int target) {
            int low = 0, high = nums.length - 1;
            int ans = -1;

            while (low <= high) {
                int mid = low + (high - low) / 2;

                if (nums[mid] == target) {
                    ans = mid;
                    low = mid + 1;
                } else if (nums[mid] < target) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

54. Count Occurrences of a Number in a Sorted Array

Understanding the Problem NEW

Count how many times a target value appears in a sorted array. E.g. target 8 in `[5,7,7,8,8,8,10]` appears 3 times.

Brute Force Approach NEW

Count occurrences with a full linear scan, incrementing a counter whenever you see the target — O(n).

Why We Optimize NEW

Once you have the first and last occurrence indices (problem 53, both computed in O(log n) via binary search), the count is simply `lastIndex - firstIndex + 1` — pure arithmetic, no additional scanning needed. This is a nice example of how solving one problem (find first/last) can make a seemingly separate problem (counting) almost trivial.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count total occurrences of target in sorted array.

Visual Diagram

```
Array: [2, 4, 4, 4, 6, 8, 10], Target: 4

       First = 1, Last = 3
       Count = Last - First + 1 = 3 - 1 + 1 = 3

       Visual:
       [2, 4, 4, 4, 6, 8, 10]
           ↑ ↑ ↑
           First=1, Last=3
           Count = 3

       Target: 5 → First=-1, Last=-1 → Count=0
```

Solution Approach

Concept: Find first and last occurrence, calculate difference.

Algorithm

```
Algorithm COUNT_OCCURRENCES(arr, target):
           first = FIRST_OCCURRENCE(arr, target)
           If first == -1: Return 0
           last = LAST_OCCURRENCE(arr, target)
           Return last - first + 1
```

Java Implementation

```
class Solution {
           public int countOccurrences(int[] nums, int target) {
               int first = findFirst(nums, target);
               if (first == -1) return 0;
               int last = findLast(nums, target);
               return last - first + 1;
           }

            private int findFirst(int[] nums, int target) {
                int low = 0, high = nums.length - 1;
                int ans = -1;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;
                     if (nums[mid] == target) {
                         ans = mid;
                         high = mid - 1;
                     } else if (nums[mid] < target) {
                         low = mid + 1;
                     } else {
                         high = mid - 1;
                     }
                 }
                 return ans;
            }

            private int findLast(int[] nums, int target) {
                int low = 0, high = nums.length - 1;

               int ans = -1;

               while (low <= high) {
                   int mid = low + (high - low) / 2;
                   if (nums[mid] == target) {
                       ans = mid;
                       low = mid + 1;
                   } else if (nums[mid] < target) {
```

```
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

55. Search in Rotated Sorted Array I

Understanding the Problem NEW

The array was originally sorted but has been 'rotated' at some pivot point (e.g. `[4,5,6,7,0,1,2]` is `[0,1,2,4,5,6,7]` rotated). Search for a target value in this rotated array, assuming all elements are unique.

Brute Force Approach NEW

Since the array isn't fully sorted anymore, a beginner might fall back to Linear Search across the whole thing — O(n), abandoning binary search entirely because 'it's not sorted.'

Why We Optimize NEW

The key realization: even after rotation, **at least one half** (left of mid, or right of mid) is always still perfectly sorted at every step. So at each step, first figure out which half is sorted by comparing the endpoints to the middle; then check whether the target could possibly lie within that sorted half's value range. If yes, recurse into that half; if no, the target must be in the other (still-rotated) half, so recurse there instead. This preserves the halving-every-step property of binary search, keeping it at O(log n) despite the array not being globally sorted.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Search target in a rotated sorted array (unique elements).

Visual Diagram

```
Array: [4, 5, 6, 7, 0, 1, 2], Target: 0

       Step 1: low=0, high=6, mid=3
       [4, 5, 6, 7, 0, 1, 2]
                 ↑
              arr[3]=7

       Check which half is sorted:
       Left half [4,5,6,7] is sorted
       Target 0 not in left half (0 < 4) → low=4
```

```
Step 2: low=4, high=6, mid=5
[4, 5, 6, 7, 0, 1, 2]
               ↑
             arr[5]=1

Right half [0,1,2] is sorted
Target 0 in right half → high=4

Step 3: low=4, high=4, mid=4
[4, 5, 6, 7, 0, 1, 2]
            ↑
          arr[4]=0
0 == 0 → Found at index 4

Result: 4
```

Solution Approach

Concept: Identify sorted half and search accordingly.

Algorithm

```
Algorithm SEARCH_ROTATED(arr, target):
           low = 0, high = n-1
           While low <= high:
               mid = (low + high) / 2
               If arr[mid] == target:
                   Return mid

               If arr[low] <= arr[mid]: // Left half sorted
                   If arr[low] <= target && target < arr[mid]:
                       high = mid - 1
                   Else:
                       low = mid + 1
               Else: // Right half sorted
                   If arr[mid] < target && target <= arr[high]:
                       low = mid + 1
                   Else:
                       high = mid - 1
           Return -1
```

Java Implementation

```
class Solution {

           public int search(int[] nums, int target) {
               int low = 0, high = nums.length - 1;

               while (low <= high) {
                   int mid = low + (high - low) / 2;
```

```
                if (nums[mid] == target) return mid;

                // Left half is sorted
                if (nums[low] <= nums[mid]) {
                    if (nums[low] <= target && target < nums[mid]) {
                        high = mid - 1;
                    } else {
                        low = mid + 1;
                    }
                }
                // Right half is sorted
                else {
                    if (nums[mid] < target && target <= nums[high]) {
                        low = mid + 1;
                    } else {
                        high = mid - 1;
                    }
                }
            }
            return -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

56. Search in Rotated Sorted Array II

Understanding the Problem NEW

Same as above, but now the rotated array may contain **duplicate** values, which can make it ambiguous which half is sorted (e.g. `[3,1,2,3,3,3,3]`).

Brute Force Approach NEW

Falling back to full Linear Search, O(n), is the 'safe' brute option once duplicates seem to break the halving trick.

Why We Optimize NEW

The fix is small: when `arr[low] == arr[mid] == arr[high]`, you genuinely can't tell which half is sorted from these three values alone — so simply shrink both boundaries inward by one (`low++, high--`) and try again, without losing correctness. In the rare worst case (all duplicates), this degrades to O(n), but in the general case it stays close to O(log n) — a good example of a small patch that restores an algorithm's applicability at the cost of a rare worst-case penalty.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Search target in rotated sorted array with duplicates.

Visual Diagram

```
Array: [2, 5, 6, 0, 0, 1, 2], Target: 0

       Step 1: low=0, high=6, mid=3
       [2, 5, 6, 0, 0, 1, 2]
                ↑
              arr[3]=0
       Found at index 3

       Array: [1, 0, 1, 1, 1], Target: 0

       Step 1: low=0, high=4, mid=2
       [1, 0, 1, 1, 1]
             ↑
           arr[2]=1
```

```
nums[low] == nums[mid] == nums[high] → Cannot decide
Shrink: low++, high--

Step 2: low=1, high=3, mid=2
[1, 0, 1, 1, 1]
      ↑
    arr[2]=1

arr[low]=0, arr[mid]=1, arr[high]=1
Left half [0] is sorted, target=0 in left
high=1

Step 3: low=1, high=1, mid=1
[1, 0, 1, 1, 1]
    ↑
  arr[1]=0
Found at index 1
```

Solution Approach

Concept: Handle duplicates by shrinking boundaries.

Algorithm

```
Algorithm SEARCH_ROTATED_II(arr, target):
           low = 0, high = n-1
           While low <= high:
               mid = (low + high) / 2
               If arr[mid] == target:
                   Return true

                 // Handle duplicates
                 If arr[low] == arr[mid] && arr[mid] == arr[high]:
                     low++; high--
                     Continue

                 If arr[low] <= arr[mid]: // Left half sorted
                     If arr[low] <= target && target < arr[mid]:
                         high = mid - 1
                     Else:

                         low = mid + 1
               Else: // Right half sorted
                     If arr[mid] < target && target <= arr[high]:
                         low = mid + 1
                     Else:
                         high = mid - 1
           Return false
```

Java Implementation

```
class Solution {
           public boolean search(int[] nums, int target) {
               int low = 0, high = nums.length - 1;

               while (low <= high) {
                   int mid = low + (high - low) / 2;

                   if (nums[mid] == target) return true;

                   // Handle duplicates
                   if (nums[low] == nums[mid] && nums[mid] == nums[high]) {
                       low++;
                       high--;
                       continue;
                   }

                   // Left half is sorted
                   if (nums[low] <= nums[mid]) {
                       if (nums[low] <= target && target < nums[mid]) {
                           high = mid - 1;
                       } else {
                           low = mid + 1;
                       }
                   }
                   // Right half is sorted
                   else {
                       if (nums[mid] < target && target <= nums[high]) {
                           low = mid + 1;
                       } else {
                           high = mid - 1;
                       }
                   }
               }
               return false;
           }
       }
```

Time & Space Complexity

Time Complexity: O(log n) average, O(n) worst | Space Complexity: O(1)

57. Find Minimum in Rotated Sorted Array

Understanding the Problem NEW

Find the minimum element in a rotated sorted array (unique elements) — this minimum is exactly the 'pivot' point where the rotation happened. E.g. in `[4,5,6,7,0,1,2]`, the minimum is 0.

Brute Force Approach NEW

Scan every element and track the smallest seen — O(n), which works but again ignores the structure.

Why We Optimize NEW

Compare the middle element to the **rightmost** element: if `arr[mid] > arr[high]`, the minimum must be somewhere to the right of mid (because the left portion is 'too high' to contain the rotation point); otherwise, the minimum is at or before mid. This single comparison rule lets you discard half the array each step, finding the pivot point in O(log n) instead of scanning everything.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum element in rotated sorted array.

Visual Diagram

```
Array: [4, 5, 6, 7, 0, 1, 2]

       Step 1: low=0, high=6, mid=3
       [4, 5, 6, 7, 0, 1, 2]
                ↑
              arr[3]=7

       arr[mid] > arr[high] → Minimum in right half
       low = mid + 1 = 4

       Step 2: low=4, high=6, mid=5
       [4, 5, 6, 7, 0, 1, 2]
                     ↑
                   arr[5]=1
```

```
       arr[mid] <= arr[high] → Minimum in left half
       high = mid = 5

       Step 3: low=4, high=5, mid=4
       [4, 5, 6, 7, 0, 1, 2]
                   ↑
                 arr[4]=0

       arr[mid] <= arr[high] → high=4

       Step 4: low=4, high=4
       Return arr[low] = 0

       Result: 0
```

Solution Approach

Concept: Compare mid with high to find pivot.

Algorithm

```
Algorithm FIND_MIN_ROTATED(arr):
           low = 0, high = n-1
           While low < high:
               mid = (low + high) / 2
               If arr[mid] > arr[high]:
                   low = mid + 1
               Else:
                   high = mid
           Return arr[low]
```

Java Implementation

```
class Solution {
           public int findMin(int[] nums) {
               int low = 0, high = nums.length - 1;

                 while (low < high) {
                     int mid = low + (high - low) / 2;
                     if (nums[mid] > nums[high]) {

                       low = mid + 1;
                   } else {
                       high = mid;
                   }
               }
               return nums[low];
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

58. Find How Many Times Array is Rotated

Understanding the Problem NEW

Figure out how many positions the original sorted array was rotated by. This turns out to be the same question as finding the *index* of the minimum element.

Brute Force Approach NEW

Scan for the index where a value is smaller than its predecessor — O(n).

Why We Optimize NEW

This directly reuses the Find-Minimum-in-Rotated-Array technique (problem 57): the index of the minimum element *is* the rotation count. No new algorithm needed — just recognize the equivalence and reuse the O(log n) binary search.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the number of rotations in a rotated sorted array.

Visual Diagram

```
Array: [4, 5, 6, 7, 0, 1, 2]

       Find minimum element index:
       Minimum = 0 at index 4
       Number of rotations = index = 4

       Verification:
       Original: [0, 1, 2, 4, 5, 6, 7]
       Rotated 4 times: [4, 5, 6, 7, 0, 1, 2]
       Index of minimum = 4

       Array: [11, 12, 15, 18, 2, 5, 6, 8]
       Minimum = 2 at index 4
       Rotations = 4

       Array: [1, 2, 3, 4, 5]
       Minimum = 1 at index 0
       Rotations = 0
```

Solution Approach

Concept: Find index of minimum element.

Algorithm

```
Algorithm COUNT_ROTATIONS(arr):
           low = 0, high = n-1
           While low < high:
               mid = (low + high) / 2
               If arr[mid] > arr[high]:
                   low = mid + 1
               Else:
                   high = mid
           Return low // Number of rotations
```

Java Implementation

```
class Solution {
           public int countRotations(int[] nums) {
               int low = 0, high = nums.length - 1;

                 while (low < high) {
                     int mid = low + (high - low) / 2;

                    if (nums[mid] > nums[high]) {
                        low = mid + 1;
                    } else {
                        high = mid;
                    }
                 }
                 return low; // index of minimum = number of rotations
           }
       }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

59. Single Element in a Sorted Array

Understanding the Problem NEW

In a sorted array, every element appears exactly twice except one, which appears once — find that lone element. E.g. `[1,1,3,3,4,4,8,8,10,10,11]` → answer `11`. Note this differs from the earlier 'find the single number' array problem because this array is *sorted*, which changes what's possible.

Brute Force Approach NEW

Since finding a 'lone' element among pairs generally uses XOR (as in the unsorted version), a beginner might XOR everything together — O(n), which works but completely ignores the sorted structure that's now available.

Why We Optimize NEW

Because the array is sorted, every proper pair sits at positions (even, odd) *before* the lone element, and (odd, even) *after* it. So you can binary search on this pattern: check whether `arr[mid]` matches its 'expected' pair partner (based on whether `mid` is even or odd); if the pattern holds, the lone element is to the right; if it's broken, the lone element is at or before mid. This exploits the sorted order to get O(log n), better than the O(n) XOR approach that's forced to look at everything.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the single element in a sorted array where every other appears twice.

Visual Diagram

```
Array: [1, 1, 2, 3, 3, 4, 4, 8, 8]

       Observation:
       Index: 0 1 2 3 4 5 6 7 8
              [1, 1, 2, 3, 3, 4, 4, 8, 8]
       Pair: (0,1) (2,3) (4,5) (6,7) 8

       Single element at index 2

       Binary Search:
```

```
       Step 1: low=0, high=8, mid=4
       [1, 1, 2, 3, 3, 4, 4, 8, 8]
                   ↑
                 arr[4]=3
       Check pair: arr[4] == arr[5] (3==3)
       Single element is to the left → high=3

       Step 2: low=0, high=3, mid=1
       [1, 1, 2, 3, 3, 4, 4, 8, 8]
             ↑
           arr[1]=1
       Check pair: arr[0] == arr[1] (1==1)
       Single element is to the right → low=2

       Step 3: low=2, high=3, mid=2
       [1, 1, 2, 3, 3, 4, 4, 8, 8]
                ↑
              arr[2]=2
       Check pair: arr[2] != arr[3] (2!=3)
       Single element found → Return 2
```

Solution Approach

Concept: Binary search using pair pattern.

Algorithm

```
Algorithm SINGLE_ELEMENT(arr):
           low = 0, high = n-1
           While low < high:
               mid = (low + high) / 2
               If mid % 2 == 1:
                   mid-- // Make mid even

               If arr[mid] == arr[mid+1]:
                   low = mid + 2
               Else:
                   high = mid
           Return arr[low]
```

Java Implementation

```
class Solution {
           public int singleNonDuplicate(int[] nums) {
               int low = 0, high = nums.length - 1;

               while (low < high) {
                   int mid = low + (high - low) / 2;

                   // Make mid even to check pairs
```

```
                if (mid % 2 == 1) {
                    mid--;
                }

                // If pair exists, single is on right
                if (nums[mid] == nums[mid + 1]) {
                    low = mid + 2;
                } else {
                    high = mid;
                }
            }
            return nums[low];
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

60. Find Peak Element

Understanding the Problem NEW

A 'peak' element is one that's bigger than both its neighbours (treating out-of-bounds neighbours as negative infinity). Find *any* peak — there might be several, you just need one. E.g. in `[1,2,3,1]`, index 2 (value 3) is a peak.

Brute Force Approach NEW

Scan left to right checking each element against both neighbours — O(n), correct but unnecessary given what binary search can offer here.

Why We Optimize NEW

The trick: if `arr[mid] < arr[mid+1]`, the array is 'still rising' at mid, which guarantees a peak exists somewhere to the right (since values can't rise forever); so you search right. Otherwise, a peak is guaranteed to exist at mid or to its left, so you search left. This guarantee — that a peak must exist in the direction you're 'still rising' toward — is precisely what makes binary search valid here, even though the array as a whole isn't sorted, giving O(log n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find a peak element (greater than its neighbors).

Visual Diagram

```
Array: [1, 2, 3, 1]

       Step 1: low=0, high=3, mid=1
       [1, 2, 3, 1]
           ↑
         arr[1]=2

       arr[1] < arr[2] → Peak on right
       low = mid + 1 = 2

       Step 2: low=2, high=3, mid=2
       [1, 2, 3, 1]
              ↑
```

```
             arr[2]=3

       arr[2] > arr[3] → arr[2] is peak
       Return 2

       Array: [1, 2, 1, 3, 5, 6, 4]
       Peaks: 2 (index 1), 6 (index 5)

       Binary Search:
       Step 1: mid=3, arr[3]=3, arr[4]=5
       3 < 5 → Peak on right

       Step 2: mid=5, arr[5]=6, arr[6]=4
       6 > 4 → Peak at index 5
```

Solution Approach

Concept: Binary search comparing mid with mid+1.

Algorithm

```
Algorithm FIND_PEAK(arr):
           low = 0, high = n-1
           While low < high:
               mid = (low + high) / 2
               If arr[mid] < arr[mid+1]:
                   low = mid + 1
               Else:
                   high = mid
           Return low
```

Java Implementation

```
class Solution {
           public int findPeakElement(int[] nums) {
               int low = 0, high = nums.length - 1;

                 while (low < high) {
                     int mid = low + (high - low) / 2;

                    if (nums[mid] < nums[mid + 1]) {
                        low = mid + 1;

                   } else {
                       high = mid;
                   }
               }
               return low;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

61. Find Square Root of a Number in log n

Understanding the Problem NEW

Find the integer square root of a number N (the largest integer whose square is ≤ N) without using a built-in sqrt function. E.g. sqrt(28) → 5, since $5^2=25 \leq 28$ but $6^2=36>28$.

Brute Force Approach NEW

Try every integer from 1 upward, squaring it and checking if it exceeds N, stopping at the last one that didn't — O(√N) time (or O(N) if you're even less careful).

Why We Optimize NEW

This is **binary search on the answer** rather than binary search on an array — a very important generalization of the technique. The 'answer space' (possible square roots from 1 to N) is monotonic: if some value k satisfies $k^2 \leq N$, every smaller value does too. That monotonic yes/no property is exactly what lets you binary search over the space of possible answers, checking the middle candidate's square against N and discarding half the remaining candidates each time — O(log N) instead of O(√N).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find integer square root of a number.

Visual Diagram

```
Number: 36

       Step 1: low=0, high=36, mid=18
       18² = 324 > 36 → high=17

       Step 2: low=0, high=17, mid=8
       8² = 64 > 36 → high=7

       Step 3: low=0, high=7, mid=3
       3² = 9 < 36 → ans=3, low=4
```

```
Step 4: low=4, high=7, mid=5
5² = 25 < 36 → ans=5, low=6

Step 5: low=6, high=7, mid=6
6² = 36 ≤ 36 → ans=6, low=7

Result: 6

Number: 10
Step 1: mid=5, 25>10 → high=4
Step 2: mid=2, 4<10 → ans=2, low=3
Step 3: mid=3, 9<10 → ans=3, low=4
Result: 3 (floor sqrt)
```

Solution Approach

Concept: Binary search on answer space.

Algorithm

```
Algorithm SQRT(x):
           If x < 2: Return x
           low = 0, high = x
           ans = 0
           While low <= high:
               mid = (low + high) / 2
               If mid * mid <= x:
                   ans = mid
                   low = mid + 1
               Else:
                   high = mid - 1
           Return ans
```

Java Implementation

```
class Solution {
           public int mySqrt(int x) {
               if (x < 2) return x;

                 int low = 0, high = x;
                 int ans = 0;

               while (low <= high) {
                   int mid = low + (high - low) / 2;

                   // Use long to prevent overflow
                   long square = (long) mid * mid;

                   if (square <= x) {
                       ans = mid;
```

```
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

62. Find the Nth Root of a Number

Understanding the Problem NEW

A generalization of the previous problem: find the integer N-th root of M (the integer x such that x^n equals M, or determine none exists).

Brute Force Approach NEW

Try every integer from 1 upward, raising it to the N-th power and comparing to M — O(M) in the worst case (or O(M^(1/N)) with a smarter loop bound).

Why We Optimize NEW

Exactly the same 'binary search on the answer' idea as square root, generalized: the candidate range is 1 to M, and whether `x^N ≤ M` is monotonic in x, so you can binary search over candidate values of x, checking each candidate's N-th power — giving roughly O(N log M) instead of a linear scan over all possible roots.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find N-th root of M using binary search.

Visual Diagram

```
N=3, M=27 (Cube root)

       Step 1: low=1, high=27, mid=14
       14³ = 2744 > 27 → high=13

       Step 2: low=1, high=13, mid=7
       7³ = 343 > 27 → high=6

       Step 3: low=1, high=6, mid=3
       3³ = 27 ≤ 27 → ans=3, low=4

       Result: 3

       N=2, M=10
       Step 1: mid=5, 25>10 → high=4
       Step 2: mid=2, 4<10 → ans=2, low=3
```

```
       Step 3: mid=3, 9<10 → ans=3, low=4
       Result: 3 (floor root)
```

Solution Approach

Concept: Binary search on answer with power check.

Algorithm

```
Algorithm NTH_ROOT(n, m):
           low = 0, high = m
           ans = 0
           While low <= high:
               mid = (low + high) / 2
               If pow(mid, n) <= m:
                   ans = mid
                   low = mid + 1
               Else:
                   high = mid - 1
           Return ans
```

Java Implementation

```
class Solution {
           public int nthRoot(int n, int m) {
               int low = 0, high = m;
               int ans = 0;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (Math.pow(mid, n) <= m) {
                         ans = mid;
                         low = mid + 1;
                     } else {
                         high = mid - 1;
                     }
                 }
                 return ans;

           }
       }
```

Time & Space Complexity

Time Complexity: O(log m * n) | Space Complexity: O(1)

63. Koko Eating Bananas

Understanding the Problem NEW

Koko has piles of bananas and H hours to eat them all; each hour she picks one pile and eats up to 'speed' bananas from it (if the pile has fewer, she finishes that pile and doesn't eat more that hour). Find the minimum integer eating speed that lets her finish within H hours.

Brute Force Approach NEW

Try every possible speed starting from 1 upward, simulating how many hours it would take at that speed, and stop at the first speed that fits within H hours. This can be O(max(piles) × n) in the worst case — trying many candidate speeds, each requiring a full simulation pass.

Why We Optimize NEW

Here's the pattern-recognition moment for this whole sub-chapter: 'find the minimum X such that condition(X) is achievable' is a classic **binary search on the answer** setup, whenever achievability is monotonic (if speed S works, every faster speed also works). So instead of trying every candidate speed one at a time, binary search the speed range from 1 to max(pile) — for each candidate speed, simulate in O(n) how many hours it takes, and narrow the search based on whether that's within H hours or not. This turns O(max(piles) × n) into O(n × log(max(piles))).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum eating speed to eat all bananas in H hours.

Visual Diagram

```
Piles: [3, 6, 7, 11], H=8

       Speed = 4:
       Hour 1: pile 3 (3/4=1 hour)
       Hour 2-3: pile 6 (6/4=2 hours)
       Hour 4-5: pile 7 (7/4=2 hours)
       Hour 6-8: pile 11 (11/4=3 hours)
```

```
        Total: 1+2+2+3 = 8 hours ✓

        Speed = 3:
        Pile 3: 1 hour
        Pile 6: 2 hours
        Pile 7: 3 hours (ceil 7/3=3)
        Pile 11: 4 hours (ceil 11/3=4)
        Total: 1+2+3+4 = 10 hours ✗

        Binary Search:
        low=1, high=11
        mid=6: total=1+1+2+2=6 ✓ → high=5
        mid=3: total=1+2+3+4=10 ✗ → low=4
        mid=4: total=1+2+2+3=8 ✓ → high=3
        Result: 4
```

Solution Approach

Concept: Binary search on eating speed.

Algorithm

```
Algorithm KOKO(piles, H):
            low = 1, high = max(piles)
            ans = high

            While low <= high:
                mid = (low + high) / 2
                hours = 0
                For each pile in piles:
                    hours += ceil(pile / mid)

                If hours <= H:
                    ans = mid
                    high = mid - 1
                Else:
                    low = mid + 1
            Return ans
```

Java Implementation

```
class Solution {
            public int minEatingSpeed(int[] piles, int h) {
                int low = 1, high = 0;
                for (int pile : piles) high = Math.max(high, pile);

                  int ans = high;
                  while (low <= high) {

                    int mid = low + (high - low) / 2;
```

```
                long hours = 0;

                for (int pile : piles) {
                    hours += (pile + mid - 1) / mid; // ceil division
                }

                if (hours <= h) {
                    ans = mid;
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log max) | Space Complexity: O(1)

64. Minimum Days to Make M Bouquets

Understanding the Problem NEW

Given flowers that bloom on specific days, find the minimum number of days needed to be able to make M bouquets, each requiring K **adjacent** already-bloomed flowers.

Brute Force Approach NEW

Try every possible day count starting from the earliest bloom day, simulating whether M bouquets are achievable by that day — potentially checking every day one at a time, each simulation costing O(n), giving O(n × maxDay) in the worst case.

Why We Optimize NEW

Same binary-search-on-the-answer pattern as Koko Eating Bananas: whether 'M bouquets are possible by day D' is monotonic in D (if it's possible by day D, it's also possible by any later day), so binary search over the range of possible days, using an O(n) feasibility check at each candidate day, instead of testing every day individually — turning a potentially huge linear scan over days into O(n × log(maxDay)).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum days to make M bouquets with K adjacent flowers.

Visual Diagram

```
Bloom: [1, 10, 3, 10, 2], M=3, K=1

       Need: 3 bouquets × 1 flower = 3 flowers
       Adjacent requirement: K=1 (any flower)

       Day 1: [1] → 1 bouquet
       Day 2: [1, 2] → 2 bouquets
       Day 3: [1, 2, 3] → 3 bouquets ✓

       Result: 3
```

```
Example 2: Bloom: [1, 10, 3, 10, 2], M=3, K=2
Need: 3 bouquets × 2 flowers = 6 flowers
But only 5 flowers total → Impossible

Binary Search:
low=1, high=10
mid=5: At day 5, flowers [1,3,2] bloom
Adjacent groups: [1] (1), [3] (1), [2] (1)
Bouquets = 0 ✗ → low=6

mid=8: At day 8, flowers [1,3,2] bloom
Still not enough → low=9

mid=9: Same → low=10

Result: -1 (Impossible)
```

Solution Approach

Concept: Binary search on days.

Algorithm

```
Algorithm MIN_DAYS(bloom, M, K):
           If M*K > n: Return -1
           low = min(bloom), high = max(bloom)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               If possible(mid, bloom, M, K):
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int minDays(int[] bloomDay, int m, int k) {
               int n = bloomDay.length;
               if (m * k > n) return -1;
               int low = Integer.MAX_VALUE, high = 0;

              for (int day : bloomDay) {
                  low = Math.min(low, day);
                  high = Math.max(high, day);
              }
```

```
            int ans = high;

            while (low <= high) {
                int mid = low + (high - low) / 2;

                if (canMakeBouquets(bloomDay, mid, m, k)) {
                    ans = mid;
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            return ans;
        }

        private boolean canMakeBouquets(int[] bloomDay, int day, int m, int k) {
            int bouquets = 0;
            int adjacent = 0;

            for (int bloom : bloomDay) {
                if (bloom <= day) {
                    adjacent++;
                    if (adjacent == k) {
                         bouquets++;
                         adjacent = 0;
                    }
                } else {
                    adjacent = 0;
                }
            }
            return bouquets >= m;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log max) | Space Complexity: O(1)

65. Find the Smallest Divisor

Understanding the Problem NEW

Given an array and a threshold, find the smallest divisor such that if every element is divided by it (rounded up) and summed, the total doesn't exceed the threshold.

Brute Force Approach NEW

Try every possible divisor starting from 1 upward, computing the resulting sum each time, and stop at the first divisor whose sum fits — potentially O(max(nums) × n).

Why We Optimize NEW

Again the same recognizable shape: 'find the smallest divisor such that condition holds' is monotonic (a bigger divisor only ever produces a smaller or equal sum), so binary search over possible divisor values from 1 to max(nums), checking the resulting sum in O(n) at each candidate — O(n × log(max(nums))) instead of a full linear scan over every possible divisor.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find smallest divisor such that sum of ceil(nums[i]/divisor) <= threshold.

Visual Diagram

```
Array: [1, 2, 5, 9], Threshold: 6

       Divisor 1: sum = 1+2+5+9 = 17 > 6 ✗
       Divisor 2: sum = 1+1+3+5 = 10 > 6 ✗
       Divisor 3: sum = 1+1+2+3 = 7 > 6 ✗
       Divisor 4: sum = 1+1+2+3 = 7 > 6 ✗
       Divisor 5: sum = 1+1+1+2 = 5 ≤ 6 ✓

       Binary Search:
       low=1, high=9
       mid=5: sum=1+1+1+2=5 ✓ → ans=5, high=4
       mid=3: sum=1+1+2+3=7 ✗ → low=4
```

```
       mid=4: sum=1+1+2+3=7 ✗ → low=5
       Result: 5
```

Solution Approach

Concept: Binary search on divisor.

Algorithm

```
Algorithm SMALLEST_DIVISOR(nums, threshold):
           low = 1, high = max(nums)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               sum = 0
               For each num in nums:
                   sum += ceil(num / mid)

               If sum <= threshold:
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int smallestDivisor(int[] nums, int threshold) {
               int low = 1, high = 0;
               for (int num : nums) high = Math.max(high, num);

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;
                     int sum = 0;

                     for (int num : nums) {
                         sum += (num + mid - 1) / mid; // ceil division
                     }
                     if (sum <= threshold) {

                      ans = mid;
                      high = mid - 1;
                   } else {
                       low = mid + 1;
                   }
              }
```

```
            return ans;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log max) | Space Complexity: O(1)

66. Capacity to Ship Packages Within D Days

Understanding the Problem NEW

Find the minimum ship capacity needed to deliver all packages (in their given order) within D days, where each day's shipment can't exceed the ship's capacity.

Brute Force Approach NEW

Try every possible capacity value starting from the largest single package weight upward, simulating how many days it takes at that capacity — up to O(sum(weights) × n) in the worst case.

Why We Optimize NEW

The same monotonic pattern again: a bigger capacity only ever needs the same or fewer days, so binary search the capacity range between max(weight) and sum(weights), simulating the day-count at each candidate capacity in O(n) — O(n × log(sum(weights))) instead of testing every possible capacity value individually. By this point in the chapter, it's worth noticing that problems 63-66 (and several more ahead) are really the *same* algorithmic template wearing different clothes.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum capacity to ship all packages within D days.

Visual Diagram

```
Weights: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Days: 5

       Capacity = 15:
       Day 1: 1+2+3+4+5=15
       Day 2: 6+7=13
       Day 3: 8=8
       Day 4: 9=9
       Day 5: 10=10
       Total days: 5 ✓

       Capacity = 14:
       Day 1: 1+2+3+4+5=15 > 14 ✗
```

```
Binary Search:
low=10, high=55
mid=32: days=4 ✓ → high=31
mid=21: days=5 ✓ → high=20
mid=15: days=5 ✓ → high=14
mid=13: days=6 ✗ → low=14
mid=14: days=5 ✓ → high=13
Result: 14
```

Solution Approach

Concept: Binary search on capacity.

Algorithm

```
Algorithm SHIP_CAPACITY(weights, days):
           low = max(weights), high = sum(weights)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               if canShip(weights, mid, days):
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int shipWithinDays(int[] weights, int days) {
               int low = 0, high = 0;
               for (int weight : weights) {
                   low = Math.max(low, weight);
                   high += weight;
               }

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                  if (canShip(weights, mid, days)) {
                      ans = mid;
                      high = mid - 1;
                  } else {
                      low = mid + 1;
                  }
             }
```

```
            return ans;
        }

        private boolean canShip(int[] weights, int capacity, int days) {
            int requiredDays = 1;
            int currentWeight = 0;

            for (int weight : weights) {
                if (currentWeight + weight > capacity) {
                    requiredDays++;
                    currentWeight = 0;
                }
                currentWeight += weight;
            }
            return requiredDays <= days;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log sum) | Space Complexity: O(1)

67. Kth Missing Positive Number

Understanding the Problem NEW

Given a sorted array of positive integers, find the k-th positive integer that's *missing* from the array. E.g. array `[2,3,4,7,11]`, k=5 → the missing numbers in order are 1,5,6,8,9,10,... so the 5th missing is 9.

Brute Force Approach NEW

Walk through positive integers 1, 2, 3, ... one at a time, checking whether each is present in the array (possibly with a linear scan or pointer), counting misses until you reach the k-th — O(n + answer size), which can be slow if k is large.

Why We Optimize NEW

At any array index i, you can directly **compute** how many numbers are missing up to that point: it's `arr[i] - (i+1)` (the difference between what the value 'should' be if nothing were missing, and its actual value). This missing-count is monotonically increasing as i increases, so you can binary search on the array index for the point where the missing-count first reaches or exceeds k, then do simple arithmetic to pin down the exact missing number — O(log n) instead of counting misses one at a time.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find kth missing positive number in a sorted array.

Visual Diagram

```
Array: [2, 3, 4, 7, 11], K=5

       Missing numbers:
       1 (missing), 5 (missing), 6 (missing), 8 (missing), 9 (missing)
            ↑           ↑          ↑           ↑           ↑
            1st         2nd        3rd         4th         5th

       5th missing = 9

       Binary Search:
       For each index i:
```

```
       missing_count = arr[i] - (i+1)

       Index 0: 2-1=1 missing
       Index 1: 3-2=1 missing
       Index 2: 4-3=1 missing
       Index 3: 7-4=3 missing
       Index 4: 11-5=6 missing

       Find first index where missing_count >= K:
       Index 3: missing=3 < 5 ✗
       Index 4: missing=6 >= 5 ✓

       ans = arr[4] - (missing_count - K) - 1
       ans = 11 - (6-5) - 1 = 9
```

Solution Approach

Concept: Binary search on missing count.

Algorithm

```
Algorithm KTH_MISSING(arr, K):
           low = 0, high = n-1
           ans = n

           While low <= high:
               mid = (low + high) / 2
               missing = arr[mid] - (mid + 1)

                 If missing >= K:
                     ans = mid
                     high = mid - 1
                 Else:
                     low = mid + 1

           Return ans + K
```

Java Implementation

```
class Solution {
           public int findKthPositive(int[] arr, int k) {
               int low = 0, high = arr.length - 1;
               int ans = arr.length;
               while (low <= high) {

                   int mid = low + (high - low) / 2;
                   int missing = arr[mid] - (mid + 1);

                   if (missing >= k) {
                       ans = mid;
```

```
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            return ans + k;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

68. Aggressive Cows

Understanding the Problem NEW

Given stall positions and a number of cows, place all cows in stalls to **maximize the minimum distance** between any two cows (so they're as spread out as possible). A classic 'maximize the minimum' optimization.

Brute Force Approach NEW

Try every possible minimum-distance value starting from 1 upward, and for each, greedily check if that many cows can be placed with at least that spacing — potentially checking every possible distance value one at a time, O(maxDistance × n).

Why We Optimize NEW

'Maximize the minimum distance such that placement is feasible' is monotonic in the opposite direction from the previous problems: a *smaller* distance is always easier to satisfy, a larger one is harder — but the feasibility still flips cleanly at one threshold, which is exactly what binary search needs. Binary search over possible distances, using an O(n) greedy check (can I fit all cows with at least this spacing?) at each candidate — O(n × log(maxDistance)) instead of testing every distance individually. This is the first 'maximize' version of the pattern, as opposed to the 'minimize' versions in problems 63-66.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Place cows in stalls with maximum minimum distance.

Visual Diagram

```
Stalls: [1, 2, 4, 8, 9], Cows: 3

       Distance = 3:
       Place cow 1: stall 1
       Place cow 2: stall 4 (distance = 3)
       Place cow 3: stall 8 (distance = 4)
       Minimum distance = 3 ✓
```

```
Distance = 4:
Cow 1: stall 1
Cow 2: stall 4 (distance=3 < 4) ✗

Binary Search:
low=1, high=8
mid=4: Can place? ✗ → high=3
mid=2: Can place? ✓ → ans=2, low=3
mid=3: Can place? ✓ → ans=3, low=4
Result: 3
```

Solution Approach

Concept: Binary search on answer (distance).

Algorithm

```
Algorithm AGGRESSIVE_COWS(stalls, cows):
           Sort stalls
           low = 1, high = stalls[n-1] - stalls[0]
           ans = 0

           While low <= high:
               mid = (low + high) / 2
               If canPlace(stalls, mid, cows):
                   ans = mid
                   low = mid + 1
               Else:
                   high = mid - 1
           Return ans
```

Java Implementation

```
class Solution {
           public int aggressiveCows(int[] stalls, int cows) {
               Arrays.sort(stalls);

                 int low = 1, high = stalls[stalls.length - 1] - stalls[0];
                 int ans = 0;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                    if (canPlace(stalls, mid, cows)) {
                        ans = mid;
                        low = mid + 1;
                    } else {

                       high = mid - 1;
                   }
```

```
            }
            return ans;
        }

        private boolean canPlace(int[] stalls, int distance, int cows) {
            int placed = 1;
            int lastPlaced = stalls[0];

            for (int i = 1; i < stalls.length; i++) {
                if (stalls[i] - lastPlaced >= distance) {
                    placed++;
                    lastPlaced = stalls[i];

                    if (placed >= cows) return true;
                }
            }
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log range) | Space Complexity: O(1)

69. Book Allocation Problem

Understanding the Problem NEW

Distribute books (given as page counts) among students, giving each student a contiguous set of books, so as to **minimize the maximum** number of pages any single student has to read.

Brute Force Approach NEW

Try every possible way of splitting the books among students (all possible partitions) and track the split that minimizes the worst student's total — combinatorially expensive, potentially exponential if done by brute enumeration.

Why We Optimize NEW

Just like Koko's eating speed, binary search on the answer: guess a maximum-pages-per-student value, then greedily check in O(n) whether that many students are enough to cover all books without exceeding the guessed maximum per student. Feasibility is monotonic (a higher 'maximum allowed' is always at least as easy to satisfy), so you binary search over the range of possible maximums — O(n × log(sum of pages)) instead of exploring every partition.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Allocate books to students with minimum maximum pages.

Visual Diagram

```
Books: [12, 34, 67, 90], Students: 2

      Allocate:
      Student 1: 12+34=46, Student 2: 67+90=157 → Max=157
      Student 1: 12, Student 2: 34+67+90=191 → Max=191
      Student 1: 12+34+67=113, Student 2: 90 → Max=113

      Minimum possible maximum = 113

      Binary Search:
      low=90 (max book), high=203 (sum)
      mid=146: Can allocate? ✓ → high=145
```

```
      mid=113: Can allocate? ✓ → high=112
      mid=101: Can allocate? ✗ → low=102
      mid=113: Can allocate? ✓ → high=112
      Result: 113
```

Solution Approach

Concept: Binary search on maximum pages.

Algorithm

```
Algorithm BOOK_ALLOCATION(books, students):
          If students > n: Return -1
          low = max(books), high = sum(books)
          ans = high

          While low <= high:
              mid = (low + high) / 2
              If canAllocate(books, mid, students):
                  ans = mid
                  high = mid - 1
              Else:
                  low = mid + 1
          Return ans
```

Java Implementation

```
class Solution {
          public int allocateBooks(int[] books, int students) {
              if (students > books.length) return -1;

                 int low = 0, high = 0;
                 for (int book : books) {
                     low = Math.max(low, book);
                     high += book;
                 }

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (canAllocate(books, mid, students)) {

                      ans = mid;
                      high = mid - 1;
                  } else {
                      low = mid + 1;
                  }
              }
```

```
            return ans;
        }

        private boolean canAllocate(int[] books, int maxPages, int students) {
            int requiredStudents = 1;
            int currentPages = 0;

            for (int book : books) {
                if (currentPages + book > maxPages) {
                    requiredStudents++;
                    currentPages = 0;
                }
                currentPages += book;
            }
            return requiredStudents <= students;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log sum) | Space Complexity: O(1)

70. Split Array - Largest Sum

Understanding the Problem NEW

Split an array into K contiguous subarrays so as to minimize the largest subarray sum — structurally identical to Book Allocation, just phrased with numbers instead of book pages.

Brute Force Approach NEW

Try every possible way to split the array into K parts and track the best (minimum of the maximums) — combinatorially expensive.

Why We Optimize NEW

This is a straight reuse of the Book Allocation technique: binary search over possible 'maximum subarray sum' values, checking feasibility (can this array be split into ≤K parts each summing to at most this value?) in O(n) at each candidate. Recognizing that a new-looking problem is a renamed version of one you've already solved is one of the most valuable pattern-matching skills in this style of problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Split array into k subarrays minimizing maximum sum.

Visual Diagram

```
Array: [7, 2, 5, 10, 8], K=2

       Possible splits:
       [7] [2,5,10,8] → max=25
       [7,2] [5,10,8] → max=23
       [7,2,5] [10,8] → max=18
       [7,2,5,10] [8] → max=24

       Minimum possible maximum = 18

       Binary Search:
       low=10 (max element), high=32 (sum)
       mid=21: Can split? ✓ → high=20
```

```
       mid=15: Can split? ✗ → low=16
       mid=18: Can split? ✓ → high=17
       mid=17: Can split? ✗ → low=18
       Result: 18
```

Solution Approach

Concept: Same as book allocation.

Algorithm

```
Algorithm SPLIT_ARRAY(nums, k):
           low = max(nums), high = sum(nums)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               If canSplit(nums, mid, k):
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int splitArray(int[] nums, int k) {
               int low = 0, high = 0;
               for (int num : nums) {
                   low = Math.max(low, num);
                   high += num;
               }

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (canSplit(nums, mid, k)) {
                         ans = mid;
                         high = mid - 1;

                    } else {
                        low = mid + 1;
                    }
              }
              return ans;
          }
```

```
        private boolean canSplit(int[] nums, int maxSum, int k) {
            int required = 1;
            int currentSum = 0;

            for (int num : nums) {
                if (currentSum + num > maxSum) {
                    required++;
                    currentSum = 0;
                }
                currentSum += num;
            }
            return required <= k;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log sum) | Space Complexity: O(1)

71. Painter's Partition Problem

Understanding the Problem NEW

Given boards of different lengths and a number of painters (each painting a contiguous set of boards, one unit length per unit time), find the minimum time to paint all boards. Yet another disguise of the same underlying problem.

Brute Force Approach NEW

Try every possible allocation of boards to painters and track the best worst-case time — combinatorially expensive.

Why We Optimize NEW

Same binary-search-on-the-answer template once more: binary search over possible 'maximum time' values, and for each candidate, greedily check in O(n) how many painters would be needed to keep every painter's total within that time. This is the third appearance of essentially the same technique (Book Allocation, Split Array, Painter's Partition) — a strong hint that once you recognize the shape ('minimize the maximum, subarray-based, monotonic feasibility'), you can solve a whole family of problems with one template.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum time to paint all boards with k painters.

Visual Diagram

```
Boards: [10, 20, 30, 40], Painters: 2

       Time per unit = 1 unit

       Same as split array:
       Possible allocations:
       [10] [20,30,40] → max=90
       [10,20] [30,40] → max=70
       [10,20,30] [40] → max=60

       Minimum time = 60
```

```
Binary Search:
low=40, high=100
mid=70: Can paint? ✓ → high=69
mid=55: Can paint? ✗ → low=56
mid=60: Can paint? ✓ → high=59
Result: 60
```

Solution Approach

Concept: Same as book allocation.

Algorithm

```
Algorithm PAINTER_PARTITION(boards, painters):
           low = max(boards), high = sum(boards)
           ans = high

           While low <= high:
               mid = (low + high) / 2
               If canPaint(boards, mid, painters):
                   ans = mid
                   high = mid - 1
               Else:
                   low = mid + 1
           Return ans
```

Java Implementation

```
class Solution {
           public int painterPartition(int[] boards, int painters) {
               int low = 0, high = 0;
               for (int board : boards) {
                   low = Math.max(low, board);
                   high += board;
               }

                 int ans = high;

                 while (low <= high) {
                     int mid = low + (high - low) / 2;

                     if (canPaint(boards, mid, painters)) {
                         ans = mid;

                          high = mid - 1;
                    } else {
                         low = mid + 1;
                    }
              }
              return ans;
```

```
        }

        private boolean canPaint(int[] boards, int maxTime, int painters) {
            int required = 1;
            int currentTime = 0;

            for (int board : boards) {
                if (currentTime + board > maxTime) {
                    required++;
                    currentTime = 0;
                }
                currentTime += board;
            }
            return required <= painters;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log sum) | Space Complexity: O(1)

72. Minimize Max Distance to Gas Station

Understanding the Problem NEW

Given existing gas station positions along a road and a number of new stations to add, place the new stations to **minimize the maximum distance** between any two adjacent stations.

Brute Force Approach NEW

Try adding stations one at a time, always splitting the currently-largest gap — a greedy simulation that can require re-scanning for the largest gap repeatedly, or trying every possible max-distance value with a slow feasibility check, giving a poor overall complexity.

Why We Optimize NEW

Binary search on the answer again, this time over possible real-valued (not just integer) maximum distances: for a candidate maximum distance D, you can compute in O(n) how many new stations would be needed to make every existing gap ≤ D (each gap of length g needs `floor(g/D)` extra stations); check if that's ≤ the available stations. Feasibility is monotonic in D (a larger D always needs fewer or equal new stations), so binary search narrows in on the smallest feasible D efficiently, even over a continuous range of real numbers.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Place k stations to minimize maximum distance between stations.

Visual Diagram

```
Stations: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], K=1

       Distances: 1,1,1,1,1,1,1,1,1
       Max distance = 1

       Place station between 1 and 10 → distances: 4.5,4.5
       Max distance = 4.5

       Binary Search:
       low=0, high=9
```

```
mid=4.5: Can place? ✓ → ans=4.5, high=4.5
mid=2.25: Can place? ✓ → ans=2.25, high=2.25
mid=1.125: Can place? ✓ → ans=1.125
Result: 1.125 (approximately)
```

Solution Approach

Concept: Binary search on distance.

Algorithm

```
Algorithm MIN_MAX_DISTANCE(stations, K):
           low = 0, high = max gap
           precision = 1e-6

           While high - low > precision:
               mid = (low + high) / 2
               If canPlace(stations, mid, K):
                   high = mid
               Else:
                   low = mid
           Return high
```

Java Implementation

```
class Solution {
           public double minmaxGasDist(int[] stations, int k) {
               int n = stations.length;
               double low = 0, high = stations[n-1] - stations[0];

                 while (high - low > 1e-6) {
                     double mid = low + (high - low) / 2;

                     int count = 0;
                     for (int i = 0; i < n - 1; i++) {
                         count += (int) ((stations[i+1] - stations[i]) / mid);
                     }

                     if (count <= k) {
                         high = mid;
                     } else {
                         low = mid;
                     }
                 }
                 return high;

           }
      }
```

Time & Space Complexity

Time Complexity: O(n * log precision) | Space Complexity: O(1)

73. Median of 2 Sorted Arrays

Understanding the Problem NEW

Given two sorted arrays, find the median of all their elements combined, ideally without actually merging them into one big array first.

Brute Force Approach NEW

Merge both arrays into one sorted array (O(n+m)) and then just read off the middle element(s) — correct and simple, but throws away an opportunity for something faster.

Why We Optimize NEW

The key insight: the median splits the combined array into a left half and right half of equal size. You can binary search on **how many elements to take from the smaller array's left side** — for each candidate split, the corresponding split point in the other array is determined by arithmetic (since the halves must add up correctly), and you check whether the boundary elements satisfy the ordering that a valid partition requires. This lets you find the median without merging anything, in O(log(min(n,m))) time — dramatically better than the O(n+m) merge-based approach, purely by binary searching over 'how much to take' rather than over array values directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find median of two sorted arrays.

Visual Diagram

```
Arr1: [1, 3], Arr2: [2]
       Sorted: [1, 2, 3] → Median = 2

       Arr1: [1, 2], Arr2: [3, 4]
       Sorted: [1, 2, 3, 4] → Median = (2+3)/2 = 2.5

       Binary Search (Partition Method):
       Arr1: [1, 3, 8, 9, 15], Arr2: [7, 11, 18, 19, 21, 25]

       We partition both arrays:
```

```
       Left half size = (5+6)/2 = 5

       Try partition in Arr1:
       part1 = 2, part2 = 3
       [1,3] | [8,9,15]
       [7,11,18] | [19,21,25]
       Left max = max(3,18) = 18
       Right min = min(8,19) = 8
       18 > 8 → need smaller partition in arr1

       part1 = 1, part2 = 4
       [1] | [3,8,9,15]
       [7,11,18,19] | [21,25]
       Left max = max(1,19) = 19
       Right min = min(3,21) = 3
       19 > 3 → need smaller partition

       part1 = 0, part2 = 5
       [] | [1,3,8,9,15]
       [7,11,18,19,21] | [25]
       Left max = max(-∞,21) = 21
       Right min = min(1,25) = 1
       21 > 1 → need smaller

       Not possible? Actually we need to ensure binary search works correctly.
```

Solution Approach

Concept: Partition method with binary search.

Algorithm

```
Algorithm MEDIAN_2_ARRAYS(A, B):
           If n > m: Swap arrays
           low = 0, high = n

           While low <= high:
               partitionA = (low + high) / 2
               partitionB = (n + m + 1) / 2 - partitionA

                 maxLeftA = partitionA == 0 ? -∞ : A[partitionA-1]
                 minRightA = partitionA == n ? ∞ : A[partitionA]
                 maxLeftB = partitionB == 0 ? -∞ : B[partitionB-1]
                 minRightB = partitionB == m ? ∞ : B[partitionB]
                 If maxLeftA <= minRightB and maxLeftB <= minRightA:

                    If (n+m)%2 == 0:
                        Return (max(maxLeftA, maxLeftB) + min(minRightA,
minRightB)) / 2
                    Else:
                        Return max(maxLeftA, maxLeftB)
```

```
                Else If maxLeftA > minRightB:
                     high = partitionA - 1
                Else:
                     low = partitionA + 1
           Return 0
```

Java Implementation

```
class Solution {
           public double findMedianSortedArrays(int[] nums1, int[] nums2) {
               int n = nums1.length, m = nums2.length;

               // Ensure nums1 is smaller
               if (n > m) {
                   return findMedianSortedArrays(nums2, nums1);
               }

               int low = 0, high = n;
               int total = n + m;

               while (low <= high) {
                   int partition1 = low + (high - low) / 2;
                   int partition2 = (total + 1) / 2 - partition1;

                   int maxLeft1 = partition1 == 0 ? Integer.MIN_VALUE :
nums1[partition1 - 1];
                   int minRight1 = partition1 == n ? Integer.MAX_VALUE :
nums1[partition1];
                   int maxLeft2 = partition2 == 0 ? Integer.MIN_VALUE :
nums2[partition2 - 1];
                   int minRight2 = partition2 == m ? Integer.MAX_VALUE :
nums2[partition2];

                   if (maxLeft1 <= minRight2 && maxLeft2 <= minRight1) {
                       if (total % 2 == 0) {
                            return (Math.max(maxLeft1, maxLeft2) +
Math.min(minRight1, minRight2)) / 2.
                       } else {
                            return Math.max(maxLeft1, maxLeft2);
                       }
                   } else if (maxLeft1 > minRight2) {
                       high = partition1 - 1;
                   } else {
                       low = partition1 + 1;
                   }
               }
               return 0;
           }
       }
```

Time & Space Complexity

Time Complexity: O(log min(n,m)) | Space Complexity: O(1)

74. Kth Element of 2 Sorted Arrays

Understanding the Problem NEW

Find the k-th smallest element across two sorted arrays combined, without necessarily merging them.

Brute Force Approach NEW

Merge both arrays (O(n+m)) and read off the k-th element directly — simple and correct, but again ignores a smarter option.

Why We Optimize NEW

Similar spirit to the median problem: you can binary search on how elements are partitioned between the two arrays to directly locate the k-th smallest without a full merge, achieving roughly O(log(min(n,m))) instead of O(n+m). (A simpler, though slightly less optimal, alternative is to repeatedly eliminate k/2 elements from whichever array has the smaller candidate at that position — also avoiding a full merge.) Either way, the lesson is the same as problem 73: full merging is a safe brute-force default, but exploiting the sorted structure of both arrays can beat it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find kth smallest element in two sorted arrays.

Visual Diagram

```
Arr1: [2, 3, 6, 7, 9], Arr2: [1, 4, 8, 10], K=5

       Merged: [1, 2, 3, 4, 6, 7, 8, 9, 10]
       5th element = 6

       Binary Search:
       Take k/2 = 2 elements from each array
       Arr1: 2,3 → compare with Arr2: 1,4
       3 < 4 → remove 2 elements from arr1

       Remaining: Arr1: [6,7,9], Arr2: [1,4,8,10]
       K = 3
```

```
Take k/2 = 1 from each
Arr1[0]=6, Arr2[0]=1
1 < 6 → remove from arr2

Remaining: Arr1: [6,7,9], Arr2: [4,8,10]
K = 2

Take 1 from each
6 > 4 → remove from arr2

Remaining: Arr1: [6,7,9], Arr2: [8,10]
K = 1 → min(6,8) = 6

Result: 6
```

Solution Approach

Concept: Binary search removing elements.

Algorithm

```
Algorithm KTH_ELEMENT(A, B, K):
           If n > m: Swap arrays
           low = max(0, K-m), high = min(K, n)

           While low <= high:
               cut1 = (low + high) / 2
               cut2 = K - cut1

                 left1 = cut1 == 0 ? -∞ : A[cut1-1]
                 right1 = cut1 == n ? ∞ : A[cut1]
                 left2 = cut2 == 0 ? -∞ : B[cut2-1]
                 right2 = cut2 == m ? ∞ : B[cut2]

               If left1 <= right2 and left2 <= right1:
                    Return max(left1, left2)
               Else If left1 > right2:
                    high = cut1 - 1
               Else:
                    low = cut1 + 1
           Return 0
```

Java Implementation

```
class Solution {
           public int kthElement(int[] arr1, int[] arr2, int k) {
               int n = arr1.length, m = arr2.length;

               if (n > m) {
```

```
                return kthElement(arr2, arr1, k);
            }

            int low = Math.max(0, k - m), high = Math.min(k, n);

            while (low <= high) {
                int cut1 = low + (high - low) / 2;
                int cut2 = k - cut1;

                int left1 = cut1 == 0 ? Integer.MIN_VALUE : arr1[cut1 - 1];
                int right1 = cut1 == n ? Integer.MAX_VALUE : arr1[cut1];
                int left2 = cut2 == 0 ? Integer.MIN_VALUE : arr2[cut2 - 1];
                int right2 = cut2 == m ? Integer.MAX_VALUE : arr2[cut2];

                if (left1 <= right2 && left2 <= right1) {
                    return Math.max(left1, left2);
                } else if (left1 > right2) {
                    high = cut1 - 1;
                } else {
                    low = cut1 + 1;
                }
            }
            return 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log min(n,m)) | Space Complexity: O(1)

75. Find the Row with Maximum Number of 1's

Understanding the Problem NEW

Given a matrix where each row is sorted (containing 0s then 1s), find the row that contains the most 1s.

Brute Force Approach NEW

Scan every cell in the matrix, counting 1s per row and tracking the max — O(rows × cols).

Why We Optimize NEW

Because each row is sorted, you can find the number of 1s in a row instantly with a binary search per row (finding the first 1, i.e. a Lower-Bound-style search) — O(log cols) per row instead of O(cols). Even better, since rows are 'sorted' relative to each other in a matrix like this, you can start scanning from the top-right corner and only move left or down as needed, visiting each row and column at most once — giving an overall O(rows + cols) traversal that beats even the per-row binary search version. This mirrors the 'staircase search' idea used again in problem 77.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find row with maximum number of 1's in a sorted binary matrix.

Visual Diagram

```
Matrix:
       Row 0: [0, 1, 1, 1]   → 3 ones
       Row 1: [0, 0, 1, 1]   → 2 ones
       Row 2: [0, 0, 0, 1]   → 1 one
       Row 3: [1, 1, 1, 1]   → 4 ones

       Result: Row 3

       Optimized Approach:
       Start from top-right corner
       Row 0: [0,1,1,1] → col=3, row=0
       Move left to first 1 at index 1 → max=3, ansRow=0
```

```
Row 1: [0,0,1,1] → col=3, row=1
At row 1, col=3: 1 → col=2: 1 → col=1: 0 (stop)
max=3 stays

Row 2: [0,0,0,1] → col=1, row=2
col=1: 0 → stop
max=3

Row 3: [1,1,1,1] → col=1, row=3
col=1: 1 → col=0: 1 → col=-1: 0
max=4, ansRow=3

Result: Row 3
```

Solution Approach

Concept: Traverse from top-right.

Algorithm

```
Algorithm MAX_ONES_ROW(matrix):
           rows = m, cols = n
           row = 0, col = n-1
           max_ones = 0, ans_row = 0

           While row < m and col >= 0:
               If matrix[row][col] == 1:
                   max_ones = col + 1
                   ans_row = row
                   col--
               Else:
                   row++
           Return ans_row
```

Java Implementation

```
class Solution {
           public int maxOnesRow(int[][] matrix) {
               int m = matrix.length, n = matrix[0].length;

              int row = 0, col = n - 1;
              int maxOnes = 0, ansRow = 0;

              while (row < m && col >= 0) {
                  if (matrix[row][col] == 1) {
                      maxOnes = col + 1;
                      ansRow = row;
                      col--;
                  } else {
```

```
                    row++;
                }
            }
            return ansRow;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m+n) | Space Complexity: O(1)

76. Search in a 2D Matrix

Understanding the Problem NEW

Search for a target in a matrix where each row is sorted, and the first element of each row is greater than the last element of the previous row — meaning the whole matrix, read row by row, is really just one big sorted sequence in disguise.

Brute Force Approach NEW

Scan every cell individually — O(rows × cols), completely ignoring that the matrix is secretly one sorted array.

Why We Optimize NEW

Because reading the matrix row-by-row produces a fully sorted sequence, you can treat the 2D matrix as a 1D sorted array of length rows×cols using simple index arithmetic (`row = idx / cols` , `col = idx % cols`) and just run ordinary binary search over that virtual array. This collapses a 2D search into the exact same O(log(rows×cols)) binary search from problem 48, just with an extra index-mapping step.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Search target in matrix where each row is sorted and first element of each row > last of previous.

Visual Diagram

```
Matrix:
       [1, 3, 5, 7]
       [10, 11, 16, 20]
       [23, 30, 34, 60]

       Target: 16

       Flattened: [1,3,5,7,10,11,16,20,23,30,34,60]

       Binary Search on flattened:
       low=0, high=11, mid=5 → arr[5]=11 < 16
       low=6, high=11, mid=8 → arr[8]=23 > 16
```

```
        low=6, high=7, mid=6 → arr[6]=16 == 16 ✓

        Map index to matrix:
        index=6 → row=6/4=1, col=6%4=2
        matrix[1][2] = 16 ✓

        Result: True
```

Solution Approach

Concept: Treat as 1D array with mapping.

Algorithm

```
Algorithm SEARCH_2D_MATRIX(matrix, target):
            m = rows, n = cols
            low = 0, high = m*n - 1

            While low <= high:
                mid = (low + high) / 2
                row = mid / n, col = mid % n
                val = matrix[row][col]

                If val == target:
                    Return true
                Else If val < target:
                    low = mid + 1
                Else:
                    high = mid - 1
            Return false
```

Java Implementation

```
class Solution {
            public boolean searchMatrix(int[][] matrix, int target) {
                int m = matrix.length, n = matrix[0].length;
                int low = 0, high = m * n - 1;

                  while (low <= high) {
                      int mid = low + (high - low) / 2;
                      int row = mid / n;

                   int col = mid % n;

                   if (matrix[row][col] == target) {
                       return true;
                   } else if (matrix[row][col] < target) {
                       low = mid + 1;
                   } else {
                       high = mid - 1;
```

```
                }
            }
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log(m*n)) | Space Complexity: O(1)

77. Search in a Row and Column Wise Sorted Matrix

Understanding the Problem NEW

Search for a target in a matrix where each row is sorted left-to-right and each column is sorted top-to-bottom — but unlike problem 76, rows aren't globally ordered relative to each other, so you can't flatten it into one sorted array.

Brute Force Approach NEW

Scan every cell — O(rows × cols), or run a binary search independently on every row — O(rows × log cols), both ignoring the column-sorted structure.

Why We Optimize NEW

Start from the **top-right corner**: if the current value is bigger than the target, the whole column below it must also be too big (column is sorted), so move left; if it's smaller, the whole row to its left is too small, so move down. Each step eliminates either a full row or a full column, so you reach the answer (or run out of matrix) in at most `rows + cols` steps — O(rows + cols), better than the row-by-row binary search approach, by exploiting *both* sorted directions at once instead of just one.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Search in matrix sorted row and column wise.

Visual Diagram

```
Matrix:
       [1, 4, 7, 11, 15]
       [2, 5, 8, 12, 19]
       [3, 6, 9, 16, 22]
       [10, 13, 14, 17, 24]
       [18, 21, 23, 26, 30]

       Target: 14

       Start from top-right: matrix[0][4] = 15
       15 > 14 → move left
       [1,4,7,11,15] → 11 < 14 → move down
```

```
[2,5,8,12,19] → 12 < 14 → move down
[3,6,9,16,22] → 16 > 14 → move left
[3,6,9,16,22] → 9 < 14 → move down
[10,13,14,17,24] → 14 == 14 ✓

Path: (0,4) → (0,3) → (1,3) → (2,3) → (2,2) → (3,2)
Result: True
```

Solution Approach

Concept: Start from top-right and move.

Algorithm

```
Algorithm SEARCH_ROW_COL_SORTED(matrix, target):
           row = 0, col = n-1

           While row < m and col >= 0:
               If matrix[row][col] == target:
                   Return true
               Else If matrix[row][col] > target:
                   col--
               Else:
                   row++
           Return false
```

Java Implementation

```
class Solution {
           public boolean searchMatrix(int[][] matrix, int target) {
               int row = 0, col = matrix[0].length - 1;

                 while (row < matrix.length && col >= 0) {
                     if (matrix[row][col] == target) {
                         return true;
                     } else if (matrix[row][col] > target) {
                         col--;
                     } else {
                         row++;
                     }
                 }
                 return false;
           }

      }
```

Time & Space Complexity

Time Complexity: O(m+n) | Space Complexity: O(1)

78. Find Peak Element in 2D Matrix

Understanding the Problem NEW

Find a 'peak' element in a 2D matrix — one that's greater than or equal to all four of its direct neighbours (up, down, left, right).

Brute Force Approach NEW

Scan every cell and check its four neighbours — O(rows × cols), correct but doesn't exploit any structure.

Why We Optimize NEW

You can binary search on **columns** instead of scanning every cell: pick the middle column, find the maximum element in that column (O(rows)), and compare it to its left and right neighbours. If it's bigger than both, it's a 2D peak — done. If a neighbour to one side is bigger, a peak is guaranteed to exist somewhere in that direction (following the same 'still rising, so a peak must exist ahead' logic as the 1D peak problem), so recurse into that half of the columns. This turns an O(rows×cols) full scan into O(rows × log(cols)) — binary searching over columns while doing a linear scan within each candidate column.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find a peak element in a 2D matrix (greater than neighbors).

Visual Diagram

```
Matrix:
       [1, 2, 3, 4]
       [5, 6, 7, 8]
       [9, 10, 11, 12]
       [13, 14, 15, 16]

       Peaks: 16 at (3,3) is a peak

       Find maximum in middle column:
       mid = 1 → column 1: [2,6,10,14] → max=14 at row 3
       Check neighbors: left=13, right=15
       14 < 15 → peak on right side
```

```
       low = mid + 1 = 2

       mid = 2 → column 2: [3,7,11,15] → max=15 at row 3
       Check neighbors: left=14, right=16
       15 < 16 → peak on right side
       low = 3

       mid = 3 → column 3: [4,8,12,16] → max=16 at row 3
       Check neighbors: left=15, right=-∞
       16 > 15 ✓ → Peak at (3,3)

       Result: (3,3)
```

Solution Approach

Concept: Binary search on columns.

Algorithm

```
Algorithm PEAK_2D(matrix):
           m = rows, n = cols
           low = 0, high = n-1

           While low <= high:
               mid = (low + high) / 2
               row = findMaxInColumn(mid)

                 left = mid == 0 ? -∞ : matrix[row][mid-1]
                 right = mid == n-1 ? -∞ : matrix[row][mid+1]

               If matrix[row][mid] > left and matrix[row][mid] > right:
                   Return (row, mid)
               Else If left > matrix[row][mid]:
                   high = mid - 1
               Else:
                   low = mid + 1
           Return (-1, -1)
```

Java Implementation

```
class Solution {
           public int[] findPeakGrid(int[][] matrix) {
               int m = matrix.length, n = matrix[0].length;

              int low = 0, high = n - 1;

              while (low <= high) {
                  int mid = low + (high - low) / 2;
                  int row = findMaxRow(matrix, mid);
```

```
                int left = mid == 0 ? -1 : matrix[row][mid - 1];
                int right = mid == n - 1 ? -1 : matrix[row][mid + 1];

                if (matrix[row][mid] > left && matrix[row][mid] > right) {
                    return new int[]{row, mid};
                } else if (left > matrix[row][mid]) {
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            return new int[]{-1, -1};
        }

        private int findMaxRow(int[][] matrix, int col) {
            int maxRow = 0;
            for (int i = 1; i < matrix.length; i++) {
                if (matrix[i][col] > matrix[maxRow][col]) {
                    maxRow = i;
                }
            }
            return maxRow;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m log n) | Space Complexity: O(1)

79. Matrix Median

Understanding the Problem NEW

Find the median of all elements in a matrix where each row is individually sorted (rows aren't necessarily related to each other, unlike problem 76).

Brute Force Approach NEW

Flatten every row into one big list, sort the whole thing, and read off the middle — O(rows×cols × log(rows×cols)) for the sort.

Why We Optimize NEW

This reuses the 'binary search on the answer' idea, but over the *range of possible values* in the matrix rather than positions: guess a candidate median value, and use each row's sortedness to count in O(log cols) how many elements in that row are ≤ the candidate (via a binary search per row, so O(rows × log cols) total per candidate). If the total count is less than half the elements, the true median must be higher, otherwise lower — binary search over the value range (not the index range) using this counting check, giving roughly O(rows × log(cols) × log(valueRange)) — much better than fully sorting every element.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find median of a row-wise sorted matrix.

Visual Diagram

```
Matrix:
       [1, 3, 5]
       [2, 6, 9]
       [3, 6, 9]

       Flattened: [1,2,3,3,5,6,6,9,9]
       Median = 5 (5th element)

       Binary Search:
       low=1, high=9
       mid=5: Count elements <= 5
       Row 0: 1,3,5 → 3
```

```
        Row 1: 2 → 1
        Row 2: 3 → 1
        Total: 5 <= floor(9/2)=4? No, 5 > 4
        ans=5, high=4

        mid=3: Count <= 3
        Row 0: 1,3 → 2
        Row 1: 2 → 1
        Row 2: 3 → 1
        Total: 4 <= 4 ✓
        ans=3, low=4

        mid=4: Count <= 4
        Row 0: 1,3 → 2
        Row 1: 2 → 1
        Row 2: 3 → 1
        Total: 4 <= 4 ✓
        ans=4, low=5

        Result: 5
```

Solution Approach

Concept: Binary search on value range.

Algorithm

```
Algorithm MATRIX_MEDIAN(matrix):
           m = rows, n = cols
           low = min(matrix), high = max(matrix)
           required = (m*n) / 2

           While low <= high:
               mid = (low + high) / 2
               count = 0

                 For each row in matrix:
                     count += upper_bound(row, mid)

                 If count <= required:
                     low = mid + 1
                 Else:
                     high = mid - 1

           Return low
```

Java Implementation

```
class Solution {
           public int matrixMedian(int[][] matrix) {
```

```
            int m = matrix.length, n = matrix[0].length;
            int low = Integer.MAX_VALUE, high = Integer.MIN_VALUE;

            for (int i = 0; i < m; i++) {
                low = Math.min(low, matrix[i][0]);
                high = Math.max(high, matrix[i][n-1]);
            }

            int required = (m * n) / 2;

            while (low <= high) {
                int mid = low + (high - low) / 2;
                int count = 0;

                for (int i = 0; i < m; i++) {
                    count += upperBound(matrix[i], mid);
                }

                if (count <= required) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            return low;
        }

        private int upperBound(int[] row, int target) {
            int low = 0, high = row.length;

            while (low < high) {
                int mid = low + (high - low) / 2;
                if (row[mid] <= target) {
                    low = mid + 1;
                } else {
                    high = mid;
                }
            }
            return low;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m log n log range) | Space Complexity: O(1)

Binary Search Summary Table

Problem	Type	Time	Space	Key Technique
Binary Search	1D	O(log n)	O(1)	Simple BS
Lower Bound	1D	O(log n)	O(1)	First >= target
Upper Bound	1D	O(log n)	O(1)	First > target
Search Insert	1D	O(log n)	O(1)	Lower bound
Floor/Ceil	1D	O(log n)	O(1)	Two separate BS
First/Last Occurrence	1D	O(log n)	O(1)	Modified BS
Count Occurrences	1D	O(log n)	O(1)	First + Last
Search Rotated I	1D	O(log n)	O(1)	Identify sorted half
Search Rotated II	1D	O(log n) avg, O(n) worst	O(1)	Handle duplicates
Min in Rotated	1D	O(log n)	O(1)	Compare mid with high
Count Rotations	1D	O(log n)	O(1)	Index of min
Single Element	1D	O(log n)	O(1)	Pair pattern
Peak Element	1D	O(log n)	O(1)	Compare mid with mid+1
Square Root	Answer	O(log n)	O(1)	BS on answer
Nth Root	Answer	O(log n * n)	O(1)	BS on answer
Koko Eating	Answer	O(n log max)	O(1)	BS on speed
Make Bouquets	Answer	O(n log max)	O(1)	BS on days
Smallest Divisor	Answer	O(n log max)	O(1)	BS on divisor
Ship Capacity	Answer	O(n log sum)	O(1)	BS on capacity
Kth Missing	Answer	O(log n)	O(1)	Missing count
Aggressive Cows	Answer	O(n log range)	O(1)	BS on distance
Book Allocation	Answer	O(n log sum)	O(1)	BS on pages
Split Array	Answer	O(n log sum)	O(1)	BS on max sum
Painter's Partition	Answer	O(n log sum)	O(1)	BS on time
Gas Station	Answer	O(n log precision)	O(1)	BS on distance
Median 2 Arrays	2 Arrays	O(log min(n,m))	O(1)	Partition method

Kth 2 Arrays	2 Arrays	O(log min(n,m))	O(1)	Partition method
Max Ones Row	2D	O(m+n)	O(1)	Top-right traversal
Search 2D Matrix	2D	O(log(m*n))	O(1)	Flattened view
Search Row/Col Sorted	2D	O(m+n)	O(1)	Top-right traversal
Peak 2D	2D	O(m log n)	O(1)	Column BS
Matrix Median	2D	O(m log n log range)	O(1)	BS on value